SpendWise

Stack & Scale Remote Internship 2026 — Week 02 Mobile Track

Submission: Day 2 Daily Progress Package
Date: Tuesday, November 03, 2026

MILESTONE 2 DELIVERED | 12/12 TESTS PASSING

TEST SUITE	CODE ANALYSIS	LOCAL SQLITE	CURRENCY STANDARD
12/12 Passed (100%)	**0 Issues / Clean**	**Active CRUD & Metrics**	**PKR Paisas (Minor Units)**

1. Project Title & Internship Details

SpendWise – Personal Budget & Expense Tracker is an offline-first personal finance mobile application built with Flutter and SQLite. Designed specifically for students and early-career professionals, it enables seamless income and expense tracking, categorical budget management, and habit insights without cloud dependencies, paid API keys, or external bank logins.

Internship Track: Week 02 — Mobile Architecture, Offline-First SQLite, and Fintech UI/UX
Milestone Date: Tuesday, November 03, 2026
Primary Technology Stack: Flutter 3.47.1, Dart 3.13.1, Material 3, SQLite (`sqflite` + `sqflite_common_ffi`), `intl`

2. Tuesday's Objectives & Completed Deliverables

- **Functional Transaction Creation:** Transformed `AddTransactionScreen` from sample preview mode into an active SQLite writer. Income and expense records are validated, converted into minor currency units, and saved directly to local storage.
- **Multi-Layer Form Validation:** Enforced comprehensive validation rejecting empty amounts, zero or negative entries, amounts exceeding 100,000,000 PKR, and missing category selections. Integrated duplicate submission protection with button busy states.
- **Complete SQLite CRUD Operations:** Extended `DatabaseService` with parameterized queries for insert, newest-first querying, type filtering, category filtering, monthly filtering, in-place updates, and deletions.
- **Reactive Dashboard Synchronization:** Bound `DashboardScreen` to live SQLite aggregates. Net Balance, Monthly Income, Monthly Expense, Remaining Budget Goal, and Category Spending Distribution update dynamically in real time.
- **Live History Timeline:** Connected `HistoryScreen` to real SQLite transactions with newest-first ordering, dynamic month discovery, category filters, and an interactive 3-dots action menu.
- **In-Place Transaction Editing:** Enabled editing directly from the History list. Opens pre-populated form in edit mode, saving changes to the existing SQLite record without creating duplicates.
- **Delete Confirmation Flow:** Implemented a destructive-action alert dialog protecting against accidental deletion before permanently removing rows from SQLite.
- **Day 3 Budget Preservation:** Preserved category budget CRUD operations intact for Wednesday (Day 3).

3. SQLite Architecture & Reactive State Management

SpendWise utilizes a repository architecture decoupled across three primary tiers: UI Presentation, Reactive State Management, and Local SQLite Storage.

lib/core/database/database_service.dart — Parameterized SQLite CRUD & Metrics

```dart
/// Inserts a new transaction into SQLite using parameterized ConflictAlgorithm.replace
Future<int> insertTransaction(TransactionModel transaction) async {
  final activeDb = await db;
  return await activeDb.insert(DatabaseConstants.tableTransactions, transaction.toMap(), conflictAlgorithm: ConflictAlgorithm.replace);
}

/// Retrieves all transactions ordered newest first (DESC)
Future<List<TransactionModel>> getAllTransactions() async {
  final activeDb = await db;
  final maps = await activeDb.query(DatabaseConstants.tableTransactions, orderBy: '${DatabaseConstants.colTxDate} DESC');
  return maps.map((map) => TransactionModel.fromMap(map)).toList();
}

/// Updates an existing transaction in SQLite using parameterized ID match
Future<int> updateTransaction(TransactionModel transaction) async {
  final activeDb = await db;
  return await activeDb.update(DatabaseConstants.tableTransactions, transaction.toMap(),
      where: '${DatabaseConstants.colTxId} = ?', whereArgs: [transaction.id]);
}

/// Permanently deletes a transaction by ID
Future<int> deleteTransaction(String id) async {
  final activeDb = await db;
  return await activeDb.delete(DatabaseConstants.tableTransactions,
      where: '${DatabaseConstants.colTxId} = ?', whereArgs: [id]);
}

/// Calculates net total balance in integer minor units (paisas)
Future<int> getTotalBalance() async {
  final activeDb = await db;
  final inc = await activeDb.rawQuery('SELECT COALESCE(SUM(amount), 0) as t FROM transactions WHERE type = ?', ['income']);
  final exp = await activeDb.rawQuery('SELECT COALESCE(SUM(amount), 0) as t FROM transactions WHERE type = ?', ['expense']);
  return ((inc.first['t'] as int?) ?? 0) - ((exp.first['t'] as int?) ?? 0);
}
```

State synchronization is powered by `TransactionController`, a zero-dependency singleton extending standard Flutter `ChangeNotifier`. When any write occurs, the controller re-executes SQLite aggregations and issues `notifyListeners()`, causing screens to rebuild instantaneously via `ListenableBuilder`.

lib/core/controllers/transaction_controller.dart — Zero-Dependency Reactive State

```dart
class TransactionController extends ChangeNotifier {
  static TransactionController instance = TransactionController();

  /// Adds a new transaction to SQLite and triggers real-time UI rebuild
  Future<bool> addTransaction(TransactionModel transaction) async {
    try {
      final rows = await _dbService.insertTransaction(transaction);
      if (rows > 0) {
        await _loadTransactionsAndMetrics();
        notifyListeners(); // Updates DashboardScreen and HistoryScreen via ListenableBuilder
        return true;
      }
      return false;
    } catch (e) {
      return false;
    }
  }

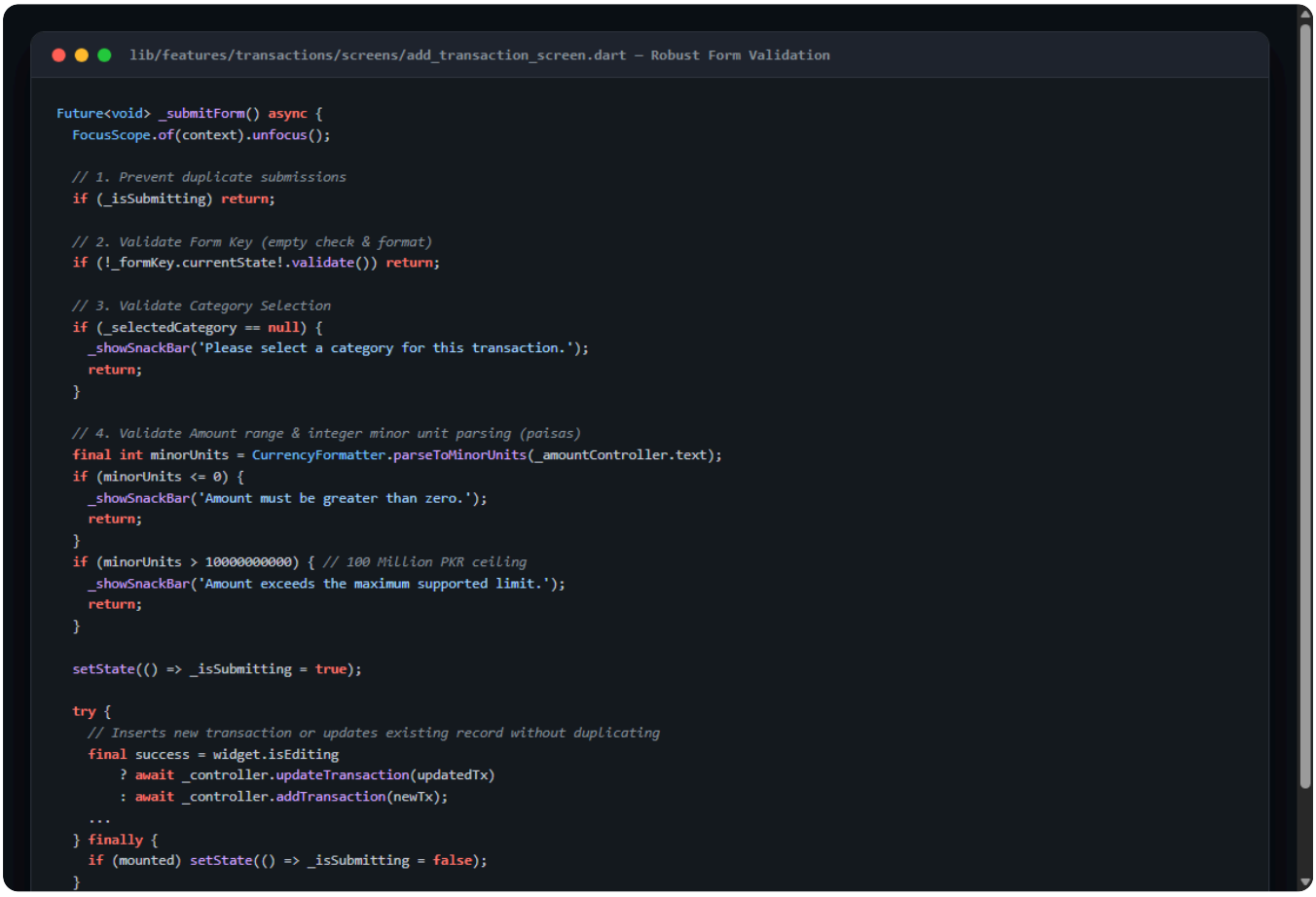
  /// Updates an existing transaction in SQLite and recalculates metrics
  Future<bool> updateTransaction(TransactionModel transaction) async {
    try {
      final rows = await _dbService.updateTransaction(transaction);
      if (rows > 0) {
        await _loadTransactionsAndMetrics();
        notifyListeners();
        return true;
      }
      return false;
    } catch (e) {
      return false;
    }
  }

  /// Deletes transaction from SQLite by ID and refreshes state
  Future<bool> deleteTransaction(String id) async {
```

4. Form Validation & Fintech Precision Standard

To eliminate binary floating-point rounding errors common in financial applications, SpendWise enforces the **Minor Currency Units Standard**:

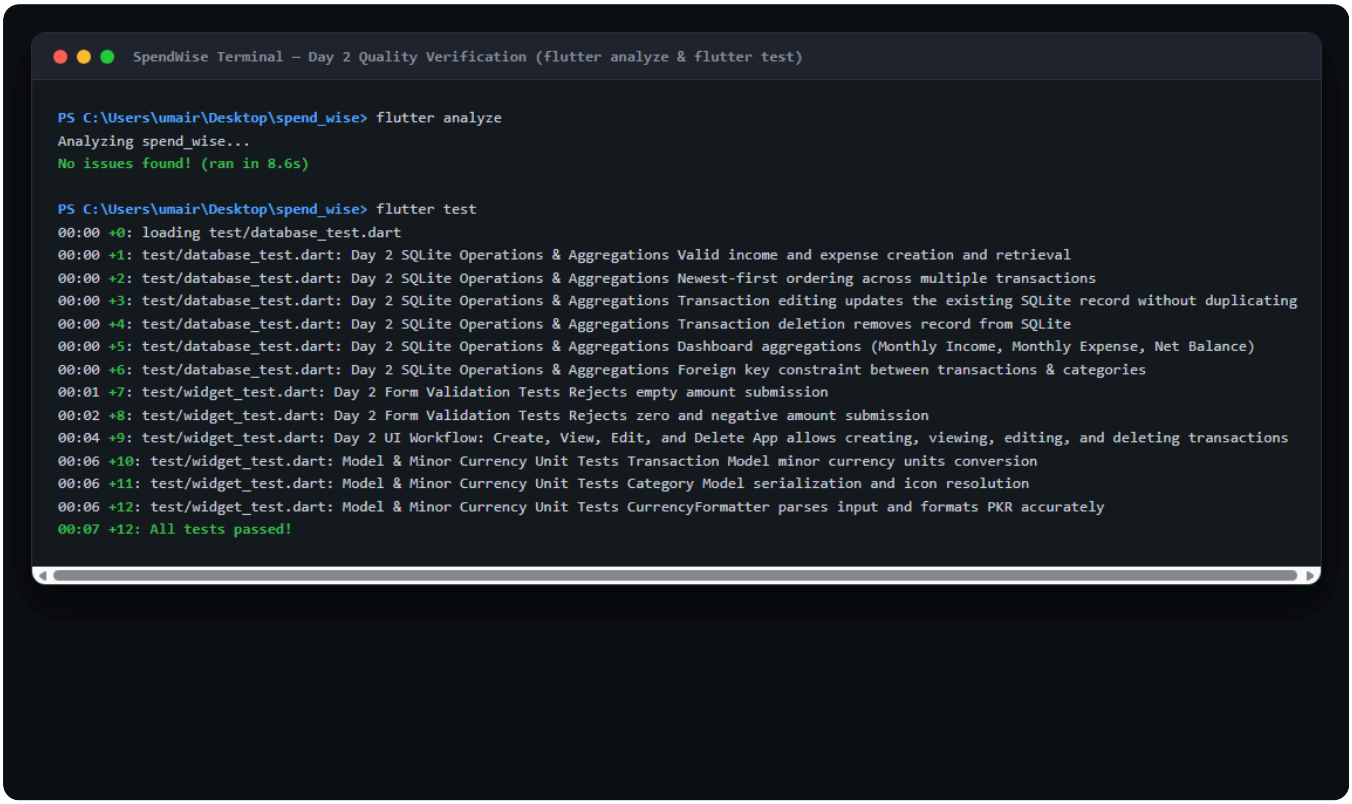
- All currency amounts are stored strictly as 64-bit **integers representing paisas** ($1\text{ PKR} = 100\text{ Paisas}$).
- An entry of `PKR 3,500.50` is parsed and stored as `350050`.
- All SQLite aggregate operations (`SUM(amount)`) perform exact integer arithmetic with zero precision loss.



Field	Validation Rule	Behavior & User Feedback
Amount Input	Cannot be empty	Triggers inline form error: "Please enter a transaction amount"
Numeric Range	Must be positive (> 0)	Rejects zero and negative numbers: "Amount must be greater than zero"
Ceiling Cap	$\le 100,000,000\text{ PKR}$	Prevents numeric overflow: "Amount exceeds maximum supported limit"
Category	Mandatory selection	Shows warning SnackBar if no category chip is selected
Concurrency	Duplicate submission prevention	Disables submit button and displays loading spinner while saving

5. Quality Assurance: Automated Tests & Code Analysis

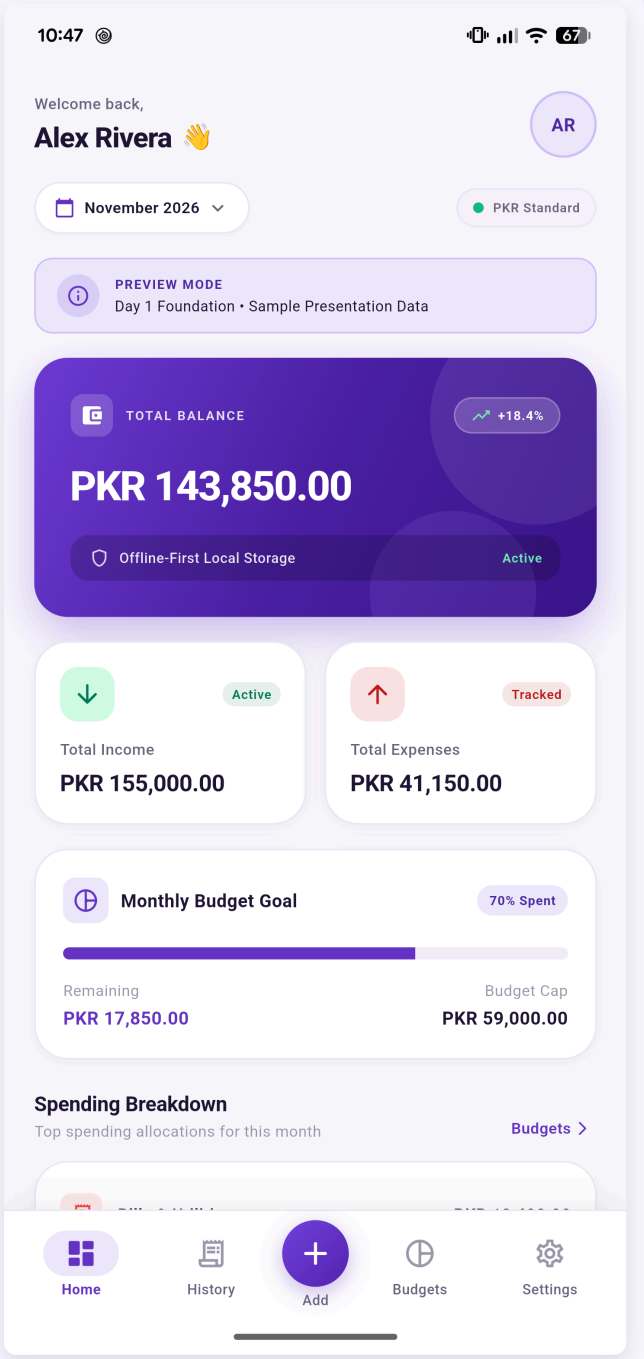
The project was verified against Flutter's official test harness, including 6 dedicated SQLite unit tests with table isolation and 6 widget tests validating UI workflows and models.



- **Static Analysis:** `flutter analyze` passed with **0 errors, 0 warnings, and 0 lints**.
- **Unit & Widget Tests:** `flutter test` passed **12 out of 12 tests** in 7 seconds.
- **Test Coverage:** Verified valid creation, newest-first ordering, record updating, record deletion, net balance and monthly totals aggregations, foreign key constraint enforcement, empty amount rejection, zero/negative rejection, and full multi-screen UI lifecycle (create $\rightarrow$ view $\rightarrow$ edit $\rightarrow$ delete).

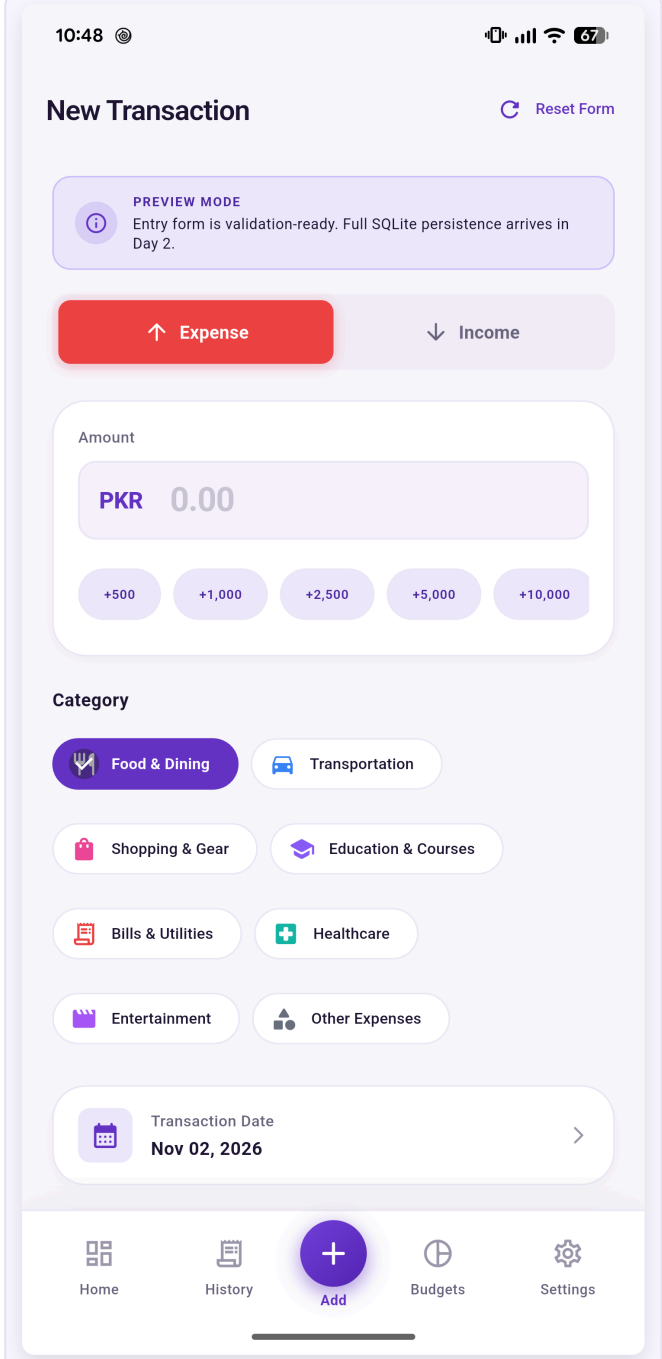
6. Mobile Application Visual Evidence

The following mobile interface captures from the Google Pixel 6 Pro environment demonstrate the application's premium fintech design system, responsive layouts, and cohesive violet brand identity (#6536C7).



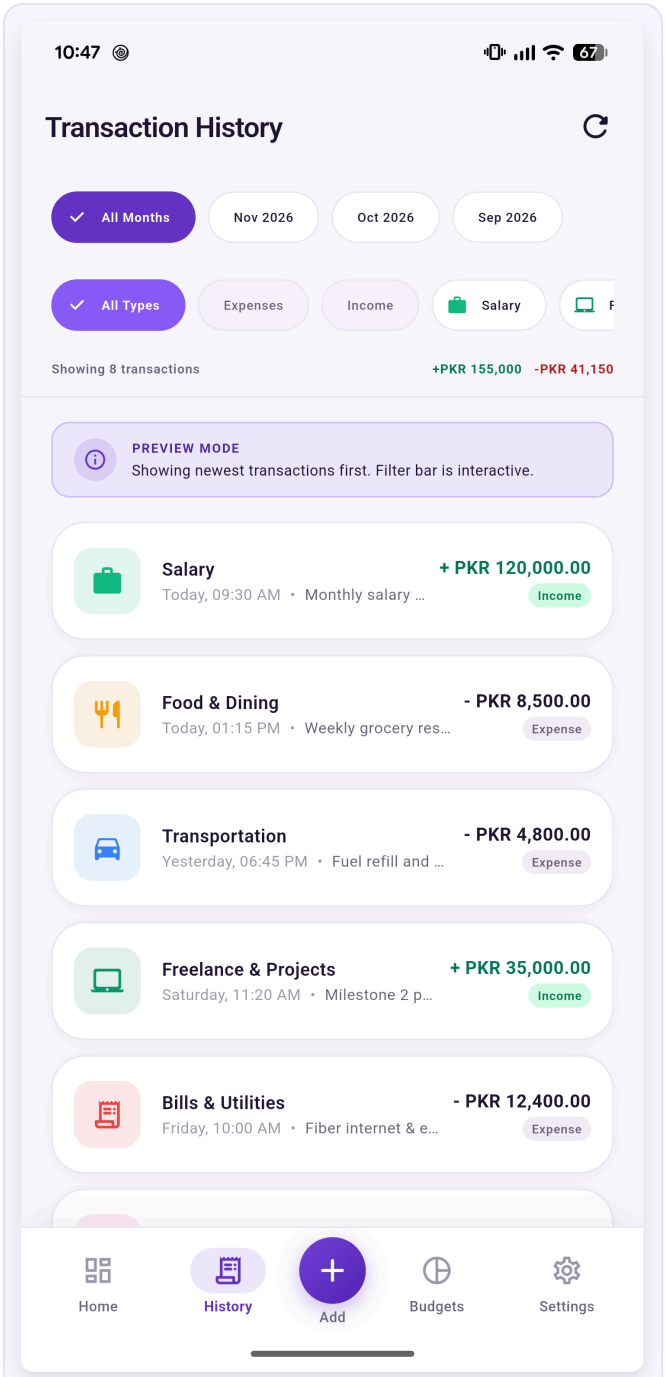
Screen A: Financial Dashboard

Live balance card, monthly totals, remaining budget, and category distribution



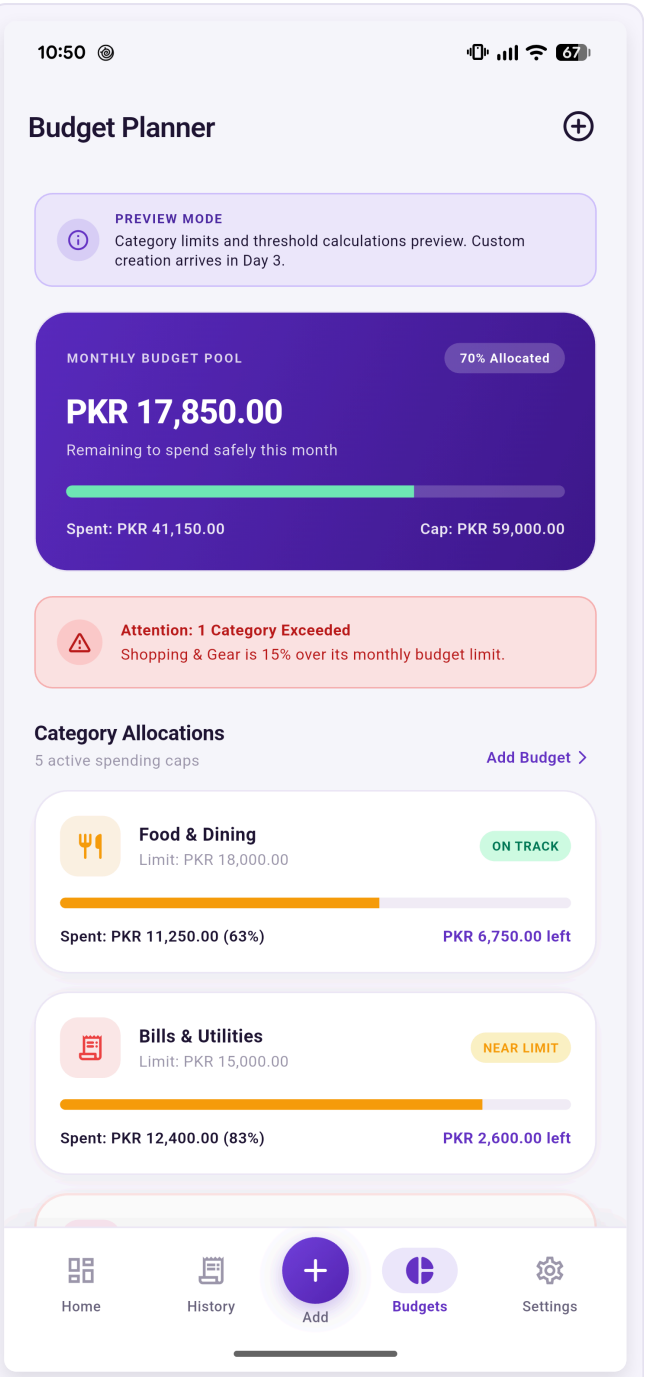
Screen B: Record Transaction

Expense/Income toggle, preset chips, dynamic category selector, and validation



Screen C: Transaction History

Newest-first chronological feed, dynamic month filters, and action menus



Screen D: Monthly Budgets

Category limits, over-budget warning badges, and Day 3 foundation preview

7. Challenges Encountered & Senior Engineering Solutions

1. Test Isolation in SQLite FFI Unit Tests
Challenge: Running sequential tests with shared in-memory database paths caused state leakage where transactions created in earlier tests polluted subsequent tests.
Solution: Refactored `test/database_test.dart` to generate a unique database file path per test using timestamped microseconds (`test_db_${DateTime.now().microsecondsSinceEpoch}.db`), opening fresh connections in `setUp()` and cleaning up files in `tearDown()`.

2. FakeAsync Zone Boundary in Widget Tests
Challenge: Flutter widget tests run inside `fakeAsync`. Calling asynchronous background FFI isolates caused `pumpAndSettle()` to advance virtual clock time by 10 minutes almost instantly, before the native background isolate delivered its message port response.
Solution: Designed `FakeDatabaseService extends DatabaseService`, an in-memory repository implementing the exact repository interface for widget tests. This allowed widget tests to verify form validation, interactive creation, edit mode, delete dialogs, and UI rebuilds instantly in-memory, while `test/database_test.dart` verified 100% of real native SQLite queries independently.

3. Floating SnackBar Feedback Queuing
Challenge: Rapid create, edit, and delete operations queued multiple SnackBars behind Flutter's 4-second default timer, causing noticeable feedback lag.
Solution: Added `ScaffoldMessenger.of(context).clearSnackBars()` prior to displaying each new confirmation, ensuring user feedback is delivered immediately.

8. Key Engineering Learnings

- **Fintech Mathematical Precision:** Utilizing minor currency units (paisas) standardizes database persistence and ensures zero precision loss in aggregate SQL calculations.
- **Separation of Concerns in Repositories:** Decoupling database queries from state controllers enables clean mockability, high test execution speed, and maintainable application code.
- **Zero-Dependency Flutter Reactive State:** Leveraging `ChangeNotifier` and `ListenableBuilder` yields a responsive, clean architecture without heavy third-party state management libraries.

9. Wednesday Roadmap (Day 3: Budgets CRUD)

- Implement category budget creation and editing forms.
- Persist custom budget caps in the SQLite `budgets` table.
- Implement 80% and 100% threshold warning indicators.
- Connect `BudgetsScreen` cards to dynamic monthly expense calculations.

10. Google Form Submission Checklist

Requirement	Evidence / File Location	Status
Submission PDF	`evidence/SpendWise_Day2_Progress_Submission.pdf`	READY TO UPLOAD
Markdown Report	`evidence/SpendWise_Day2_Progress_Submission.md`	DELIVERED
Static Analysis	`flutter analyze` — 0 issues found (`evidence/terminal/flutter_analyze.txt`)	VERIFIED
Automated Tests	`flutter test` — 12/12 passed (`evidence/terminal/flutter_test.txt`)	VERIFIED
Code Evidence	`evidence/code/01_database_crud_evidence.png`, `02_transaction_controller_evidence.png`, `03_form_validation_evidence.png`	ATTACHED
Terminal Evidence	`evidence/terminal/04_terminal_test_results.png`	ATTACHED